

Natural Language Autoencoders Read and Steer Vision-Language-Action Policy Backbones

Anonymous Author(s)

Affiliation
email@institution.edu

May 17, 2026

Abstract

Vision-language-action (VLA) policies such as NVIDIA’s GR00T expose a high-dimensional Qwen3-VL backbone hidden state $\mathbf{h}$ that is causally upstream of the action head. There is no standard way to read $\mathbf{h}$ out as natural language or to write back into it from language. We adapt the Natural Language Autoencoder (NLA) recipe [1] to this setting and release **nla-groot**, an open-source implementation that (i) extracts per-token activations from GR00T’s backbone, (ii) trains a paired *activation verbalizer* AV ($\mathbf{h} \rightarrow y$) and *activation reconstructor* AR ($y \rightarrow \hat{\mathbf{h}}$) with joint supervised fine-tuning, with an optional GRPO phase that uses AR as a learned reconstruction reward, and (iii) injects AR’s output back into the live policy server as a backbone steering perturbation in LIBERO closed-loop simulation. We propose a three-axis evaluation protocol that combines offline reconstruction, a multimodal LLM grounding judge, and a closed-loop steerability A/B test. We report a pilot study on LIBERO Goal in which our V3 checkpoint attains a retrieval margin of 0.124 and 100% caption *appropriateness*, but only 8.3% anti-template specificity, 41.7% grounding, and a zero correct-minus-wrong steering gap on the single task tested. Reconstruction metrics alone would call this run a success; the full protocol does not. We propose this protocol, together with the “shorthand template collapse” failure mode it surfaces, as a prerequisite for future NLA-style interpretability work on VLA policies.

1 Introduction

Recent vision-language-action (VLA) models pair a vision-language backbone with a diffusion or flow-matching action head and act end-to-end on physical robots [2–4]. As these models scale, a question becomes pressing: what does the policy represent internally about the task it is about to execute, in a form that a human can read and modify? Interpretability tooling for language models, including probes [5, 6], sparse autoencoders [7, 8], and activation steering [9, 10], has not yet been adapted to the policy setting, where downstream behavior is itself an evaluation signal.

The NLA recipe applied to a VLA.. Fraser-Taliente et al. [1] show that an LM hidden state can be encoded into text and decoded back: a small *activation verbalizer* (AV) is trained to caption an injected activation, and a paired *activation reconstructor* (AR) is trained to invert the caption back into activation space. This produces both an interpretability artifact (the caption) and a causal handle (the reconstructed vector). We ask whether the same recipe transfers to a VLA backbone, in which the hidden state is intended to carry scene and task information that the action head consumes.

Contributions..

1. **System.** An open-source implementation (`nla-groot`) that extracts GR00T backbone activations (Qwen3-VL, 16-layer truncation, $d=2048$), labels them with a multimodal teacher, jointly trains AV and AR on top of a Qwen3-4B-Instruct base, and exposes AR’s output as a live backbone steering vector in the official GR00T LIBERO policy server (§3).
2. **Evaluation protocol.** Three complementary axes: offline reconstruction (FVE, cosine, retrieval), a multimodal LLM judge on cached frames (axes B and C), and a closed-loop sim A/B test with matched, mismatched, and null caption arms. Results are aggregated into a single PASS/WARN/FAIL scorecard (§4).
3. **Pilot study.** On a LIBERO 4-suite combined corpus, our best checkpoint passes reconstruction metrics but fails grounding and steering. We characterize this “shorthand template collapse” failure mode, show that retrieval margin and reconstruction cosine alone are insufficient as a success criterion, and document a train-deploy distribution gap that limits behavioral steering (§5).

Thesis.. *NLA-style autoencoding is a viable approach to VLA interpretability, but only behind a multi-axis protocol; reconstruction metrics in isolation are misleading.*

2 Background and related work

Vision-language-action policies.. GR00T [2] couples a Cosmos-Reason2-2B / Qwen3-VL backbone with a diffusion action head. Only the first `SELECT_LAYER=16` decoder layers are retained at deployment, so the policy reads a 2048-dimensional hidden state at layer 16. Related VLAs include RT-2 [3], OpenVLA [4], and Octo [11]. We focus on GR00T because its backbone-head split is explicit and its checkpoints are publicly available.

Activation interpretability.. Linear probes recover task-relevant subspaces [5], and sparse autoencoders factor activations into approximately monosemantic features [7, 8, 12]. NLA [1] complements these methods by producing free-text explanations $y = \text{AV}(\mathbf{h})$ together with a faithfulness anchor $\hat{\mathbf{h}} = \text{AR}(y)$. `nla-groot` ports this recipe to a VLA backbone.

Activation steering.. Activation editing has produced controllable behavior in language models via linear directions [9, 10, 13, 14]. Behavioral steering of robot policies is less studied, and closed-loop simulation is the appropriate validation setting when the model is a policy. We treat AR’s output as a candidate steering vector and measure whether it changes downstream behavior on LIBERO.

Robotic simulation benchmarks.. LIBERO [15] is a suite of MuJoCo-based manipulation tasks with public assets. GR00T provides posttrained checkpoints per LIBERO sub-suite (e.g. `libero_goal`), which we use as the closed-loop ground truth.

3 Method

3.1 Data: extraction and labeling

Per-token backbone activations are captured by a forward hook on the GR00T `backbone_features` tensor, before the post-VL LayerNorm and the cross-attention to the action head (the layer spec is documented in `src/nla/layer_spec.py`). For each LeRobot episode and step we record $\mathbf{h}_i \in \mathbb{R}^{2048}$ at three position types (text, image-patch, image-summary; see Fraser-Taliente et al. [1]). A multimodal

teacher (GPT-class) is given the raw camera frame(s), instruction, and token-position metadata, and emits a structured bullet-point caption that we treat as the gold target for supervised training.

A per-corpus scalar α is estimated as the 75th percentile of $\|\mathbf{h}\|_2$ and used for both volumetric calibration of injection and as the unit of MSE in AR training. For the LIBERO 4-suite combined corpus $\alpha \approx 203.98$.

3.2 Architecture: AV and AR

Both AV and AR are LoRA fine-tunes (rank 32, $\alpha = 64$, full Qwen attention/MLP target set, bfloat16) of Qwen/Qwen3-4B-Instruct-2507.

AV: activation $\rightarrow$ text.. The prompt template contains a single `<|act_slot|>` token. At forward time we tokenize the prompt, embed the IDs, then overwrite the slot embedding with $\alpha \cdot \text{normalize}(\mathbf{W}_p \mathbf{h})$ where $\mathbf{W}_p \in \mathbb{R}^{h \times 2048}$ is a learnable projector. Loss is cross-entropy on the bullet caption.

AR: text $\rightarrow$ activation.. The base LM is physically truncated to its first 16 layers (matching GR00T’s `SELECT_LAYER`). A linear head reads the last non-pad hidden state of the prompt “*Summary of the following text: <text> <summary>*” and predicts $\hat{\mathbf{h}}/\alpha \in \mathbb{R}^{2048}$. Training loss is mean-squared error in α -scaled space, plus an InfoNCE contrastive term with mined hard negatives (top- k cosine, $k=4$).

3.3 Training

Joint SFT.. AV and AR are updated on the same batches, so that AR sees AV’s gold-text targets and AV cannot drift independently. The main configuration (run `libero_4suite_v3`) uses 15,000 steps with batch size 4, learning rate 10^{-4} , 500-step warmup, cosine schedule, `ar_av_mix_max = 0.4` (scheduled mixing of AR-on-AV-sampled-text past 30% of training, used to close the train-deploy text gap), and held-out fraction 0.05 with an episode-level split.

Optional GRPO phase.. After SFT, AV can be further trained by GRPO with the frozen AR as the reward model: for each $\mathbf{h}$ we sample K captions, score each by $-\|\mathbf{h}/\alpha - \text{AR}(y)\|^2$, and apply a within-group advantage policy gradient with a Schulman-k3 KL to a frozen reference AV.

3.4 Steering: language $\rightarrow$ live policy

At inference, a user supplies a short bullet caption y . We compute $\hat{\mathbf{h}} = \text{AR}(y)$ and inject it into the live GR00T policy server by replacing or blending the backbone token features at one of three placements: a single `image_patch` token, *all* image-patch tokens (`image_patch_all`), or the image-summary token. A scalar `blend` $\in [0, 2]$ controls intensity. The injection is wired through `src/nla/steering/sim_policy_wrapper.py` so the official LIBERO sim client sees no difference apart from modified backbone features on every `get_action` call.

4 Evaluation protocol

We evaluate each checkpoint on three independent axes. A single scorecard (`scripts/eval/build_v3_scorecard.py`) emits PASS/WARN/FAIL per metric against pre-registered thresholds.

4.1 Axis 1: Offline reconstruction and discriminability

- **Closed-loop cosine:** $\cos(\mathbf{h}, \text{AR}(\text{AV}(\mathbf{h})))$, computed without teacher forcing, in order to penalize AV captions that AR cannot invert.

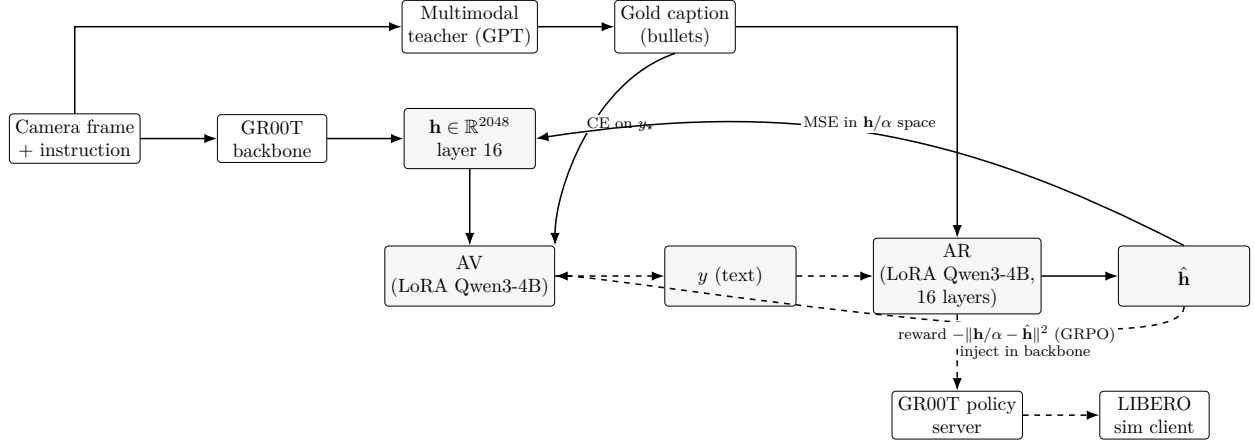


Figure 1: Data flow in `nla-groot`. Solid arrows are training paths; dashed arrows are inference / steering paths. AR doubles as the GRPO reward model and as the source of the live backbone steering vector.

- **Retrieval margin** and **retrieval@K**: matched versus cross-pair cosine on $\text{AR}(\text{AV}(\mathbf{h}))$ against a held-out pool. These metrics detect non-discriminative $\hat{\mathbf{h}}$, i.e. template collapse.

4.2 Axis 2: Multimodal LLM judge

The script `scripts/eval/llm_judge_av_captions.py` re-renders the original frame, supplies the multimodal judge with both the frame and the caption, and scores three rubrics: (A) format and length, (B) *grounding* (does the caption describe what is in the image, rather than a generic template), and (C) *appropriateness* (does the caption have the expected type for this position). Gold (teacher) captions on the same frames define a calibration ceiling. We report both gold and AV pass rates and the gap between them.

4.3 Axis 3: Closed-loop steerability A/B

A closed-loop steerability A/B test (`scripts/eval/closed_loop_sim_ab.py` and `scripts/eval/steerability_eval.py`) runs LIBERO Goal episodes on the official GR00T checkpoint under three caption arms per task: *baseline* (no steering), *correct* (caption that matches the scene and instruction), and *wrong* (a contradictory caption). The primary metric is $\Delta_{cw} = \text{succ}_{\text{correct}} - \text{succ}_{\text{wrong}}$; we register $\Delta_{cw} \geq +5\text{pp}$ as the PASS threshold. Per-object displacement and minimum end-effector distance are logged for failure analysis.

5 Experiments and results

5.1 Setup

The main checkpoint, `libero_4suite_v3`, is trained on a combined LIBERO Goal/Spatial/Object/10 activation corpus with the hyperparameters in §3. Closed-loop simulation is run against `nvidia/GR00T-N1.7-LIBERO/libero_goal` (embodiment `LIBERO_PANDA`) on the single task `put_the_bowl_on_the_plate`, with three seeds $\{0, 1, 2\}$, `max_episode_steps`= 300, and eight action steps per chunk.

5.2 Scorecard

Table 1 reproduces the V3 scorecard verbatim from `data/sft/libero_4suite_v3/v3_scorecard.json`.

Table 1: V3 scorecard (`libero_4suite_v3`). Thresholds are pre-registered in `build_v3_scorecard.py`. Required metrics determine the *overall* verdict; the overall verdict is **FAIL**.

Metric	Value	Pass-thr	Warn-thr	Verdict
Required				
<code>retrieval_margin</code>	0.124	0.05	0.02	PASS
<code>judge_grounding_specific_pct</code>	0.417	0.55	0.40	WARN
<code>judge_anti_template_specific_pct</code>	0.083	0.50	0.30	FAIL
Informative				
<code>judge_appropriateness_pct</code>	1.000	0.80	0.60	PASS
<code>retrieval@1</code>	0.039	0.25	0.15	FAIL
<code>retrieval@5</code>	0.124	0.55	0.40	FAIL
<code>closed_greedy_cosine</code>	0.388	0.55	0.40	FAIL
<code>sim_correct_success</code>	0.000	0.30	0.15	FAIL
<code>sim_correct_minus_baseline_floor</code>	-1.00	-0.10	-0.30	FAIL
<code>sim_wrong_minus_baseline</code>	-1.00	0.00	0.05	PASS*
<code>sim_correct_minus_wrong</code>	0.000	0.05	0.00	WARN

*Lower-is-better; trivially passes because *both* arms have zero success.

Table 2: Multimodal judge: gold (teacher) vs. AV captions, pass rates on a 24-row held-out sample per dataset. Numbers from `data/eval/steerability_v1_vs_v3/av_metrics.json`.

AR	Held-out	Grounding	Approp.	Template-distinguish.
<code>libero_goal_pilot</code>	gold	0.833	0.917	0.833
	AV	0.333	0.667	0.500
<code>libero_4suite_v3</code>	gold	0.833	1.000	0.833
	AV	0.583	1.000	0.500
<code>libero_4suite_v3 @ 4-suite</code>	gold	0.750	1.000	0.500
	AV	0.417	1.000	0.083

5.3 AV captions versus teacher (judge)

Table 2 reports judge pass rates on a 24-row held-out sample, taken verbatim from `av_metrics.json`. The gap between AV and the gold ceiling is the key result: 25 to 50 percentage points on grounding, and 33 to 42 percentage points on anti-template specificity. The gold ceiling itself is 75 to 83%, which sets an upper bound that a competitive AV should approach.

5.4 Closed-loop steerability

We sweep three steering prompts and two AR variants (`libero_goal_pilot` and `libero_4suite_v3`) across three seeds (Table 3, distilled from `data/eval/steerability_v1_vs_v3/metrics.json`). The baseline policy solves the task on every seed (`success_any_rate`= 1.0). Every steering condition prevents task completion (`success_any_rate`= 0.0). Steering therefore has a consistent behavioral effect, but the effect is symmetric: the correct prompt (`bowl_plate`) and the contradictory prompts (`wine_rack`, `bowl_stove`) both block the task. This gives $\Delta_{cw} = 0$ and rules out any semantic claim from this pilot. The per-object displacement and end-effector traces in Figure 2 show that all steered conditions collapse the bowl’s displacement to approximately 0.07m, compared with 0.145m at baseline. In other words, steering reduces motion uniformly rather than redirecting it toward the prompted object.

Table 3: LIBERO Goal closed-loop steerability on `put_the_bowl_on_the_plate`, three seeds. “Target winner rate” is the fraction of seeds in which the maximum-displaced tracked body was the one named by the steer prompt.

Condition	succ _{any}	Target displacement (m)	Target winner rate
baseline	1.00	0.145 (bowl)	1.00
steer_bowl_plate (v1 AR)	0.00	0.072 (bowl)	1.00
steer_wine_rack (v1 AR)	0.00	0.066 (bottle)	0.00
steer_bowl_stove (v1 AR)	0.00	0.077 (bowl)	1.00
steer_bowl_plate (v3 AR)	0.00	0.069 (bowl)	0.67
steer_wine_rack (v3 AR)	0.00	0.070 (bottle)	0.00
steer_bowl_stove (v3 AR)	0.00	0.072 (bowl)	1.00



(a) Per-object displacement.



(b) Minimum end-effector distance.

Figure 2: Per-tracked-body summaries across conditions on `put_the_bowl_on_the_plate`. Baseline (left bar in each group) is the only condition with task-typical bowl displacement; every steering arm dampens motion approximately uniformly.

5.5 Discussion of results

We summarize three findings.

(i) Reconstruction passes; behavior does not.. The retrieval margin and appropriateness rate, considered in isolation, would read as a successful training run. The grounding and anti-template scores from the judge, together with the symmetric LIBERO-Goal failure, contradict this reading. The protocol therefore serves its purpose: reporting reconstruction metrics alone would have led to an overclaim.

(ii) Shorthand template collapse persists in V3.. The 8.3% anti-template rate on the 4-suite holdout means that AV captions are largely indistinguishable from a corpus-mean template. This is the same failure mode reported for the earlier `droid_100ep_v2_nce` run (V2 post-mortem in `docs/sft_plan/06_v2_postmortem_v3_rerun.md`). The V3 mitigations (`ar_av_mix_max`, InfoNCE hard negatives, and an episode-level split) narrow the gap but do not close it.

(iii) Steering changes behavior, but not yet semantically.. Every steered condition blocks task completion, so the perturbation clearly has effect, but the identity of the prompt does not appear to determine what the policy attends to. We attribute this to two factors: (a) the train-deploy distribution gap for AR, which is trained on gold-style bullet captions and deployed on hand-written ones; and (b) the `image_patch_all` placement at `blend=1.0`, which is a coarse intervention, consistent with the open-loop probe in `scripts/eval/nla_steer_quant_probe.py`.

6 Limitations

- **Single task, small sample.** The closed-loop simulation numbers above cover three seeds on one LIBERO Goal task. We treat them as a feasibility study rather than a benchmark. A multi-task LIBERO sweep with per-prompt affordance scoring would be required before any quantitative claim about semantic steering.

- **Teacher and $\mathbf{h}$ confound.** The multimodal teacher sees the original frame and may describe scene attributes that are only weakly present in $\mathbf{h}$. The SFT loss therefore does not correspond exactly to “what $\mathbf{h}$ encodes”. This is inherited from the NLA recipe and motivates the judge’s grounding axis.
- **AR train-deploy gap.** AR is trained on gold-style bullet captions, whereas user-written steering prompts are out-of-distribution. The `ar_av_mix_max` schedule reduces but does not eliminate this gap.
- **Single backbone, single layer.** Activations are extracted at layer 16 (`SELECT_LAYER`) before the post-VL LayerNorm. Other choices (`v1_embeds`, DiT mid-block) are documented in `src/nla/layer_spec.py` but are not ablated here.
- **Judge as oracle.** The grounding judge is itself a large multimodal model and may be miscalibrated. We partially address this by pairing every AV score with a gold score on identical frames.

7 Discussion and outlook

`nla-groot` shows that the NLA recipe is implementable on a production VLA backbone, and that the resulting checkpoint can be placed in the loop of a public closed-loop simulator. The principal contribution of this paper is the evaluation protocol rather than any single result. A multi-axis scorecard with pre-registered thresholds and a behavioral arm distinguishes apparently successful runs from genuinely successful ones.

A natural next step is the GRPO phase against the AR-as-reward signal, for which the supporting code is in place (`scripts/training/run_grpo.py`) but no results are reported here. GRPO with a judge-blended reward should directly address template collapse, since judge rewards penalize generic captions in a way that the supervised cross-entropy objective does not.

Reproducibility

Code: <https://github.com/<anonymized>/nla-groot> (commit pinned at submission). Pinned environment, full hyperparameters and canonical commands appear in App. A. Seeds: SFT and steerability rollouts both default to `seed=0`; the closed-loop sim numbers reported here use seeds `{0, 1, 2}`.

References

- [1] Connor Fraser-Taliente et al. Natural language autoencoders produce unsupervised explanations of LLM activations. *Transformer Circuits Thread*, 2026. Anthropic interpretability dispatch.
- [2] NVIDIA. GR00T-N1: An open foundation model for generalist humanoid robots. *arXiv preprint*, 2025. Public checkpoints: `nvidia/GR00T-N1.7-3B`, `nvidia/GR00T-N1.7-LIBERO`.
- [3] Anthony Brohan et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023.
- [4] Moo Jin Kim et al. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning (CoRL)*, 2024.
- [5] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. In *ICLR Workshop*, 2017.
- [6] Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1), 2022.

- [7] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.
- [8] Trenton Bricken et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023.
- [9] Alexander Matt Turner, Lisa Thiergart, David Udell, Gavin Leech, Ulisse Mini, and Monte MacDiarmid. Activation addition: Steering language models without optimization. *arXiv preprint arXiv:2308.10248*, 2023.
- [10] Andy Zou et al. Representation engineering: A top-down approach to AI transparency. *arXiv preprint arXiv:2310.01405*, 2023.
- [11] Octo Model Team. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- [12] Adly Templeton et al. Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. *Transformer Circuits Thread*, 2024.
- [13] Sheng Liu, Haotian Ye, Lei Xing, and James Zou. In-context vectors: Making in context learning more effective and controllable through latent space steering. In *International Conference on Machine Learning (ICML)*, 2024.
- [14] Nina Panickssery, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Matt Turner. Steering llama 2 via contrastive activation addition. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [15] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.

A Reproducibility appendix

Hardware.. A single NVIDIA H100 (80GB) is sufficient for joint SFT at `batch_size=4`, `grad_accum_steps=1`, `gradient_checkpointing=True`. LIBERO sim is CPU-rendered (MUJOCO_GL=osmesa).

Software.. The pinned package set used for the reported runs is in `paper/repro/requirements.txt`. Notable versions: `torch==2.7.1+cu128`, `transformers==4.57.3`, `peft==0.17.1`, `accelerate==1.13.0`, `datasets==3.6.0`, `deepspeed==0.17.6`, `safetensors==0.7.0`, `numpy==1.26.4`, Python 3.10.12.

Checkpoint identifiers.. Backbone: `nvidia/GR00T-N1.7-3B` (canonical; we extract from the LIBERO-finetuned variants `nvidia/GR00T-N1.7-LIBERO/libero_{suite}`). Base LM for AV/AR: `Qwen/Qwen3-4B-Instruct-2507`.

Canonical commands.. See `paper/repro/canonical_commands.sh` for the SFT, GRPO, extraction, scorecard, and steerability invocations used in this paper.

Hyperparameters.. The full SFT config snapshot is at `data/sft/libero_4suite_v3/config.json`; we reproduce its non-default entries in Table 4.

Table 4: Non-default SFT hyperparameters for `libero_4suite_v3`.

Key	Value
<code>alpha</code>	203.977
<code>batch_size</code>	4
<code>grad_accum_steps</code>	1
<code>learning_rate</code>	10^{-4}
<code>warmup_steps</code>	500
<code>total_steps</code>	15,000
<code>av_weight / ar_weight</code>	1.0 / 1.0
<code>ar_contrastive_weight</code>	0.5
<code>ar_nce_hard_negatives_per_anchor</code>	4
<code>nce_temperature</code>	0.1
<code>ar_av_mix_max</code>	0.4
<code>ar_av_mix_warmup_frac</code>	0.3
<code>gradient_checkpointing</code>	true
<code>eval_closed_loop</code>	true
<code>closed_loop_temperatures</code>	{0.0, 0.7}
<code>balance_position_mix</code>	true
<code>split_by</code>	episode
<code>held_out_fraction</code>	0.05

Scorecard schema.. Per-metric thresholds and JSON sources are defined in `scripts/eval/build_v3_scorecard.py`; the machine-readable scorecard for this paper is `data/sft/libero_4suite_v3/v3_scorecard.json`.